llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5_exact · claude-haiku-4-5-20251001 · FASTER · RIGHT = attempt 4 of 49, rejected by the gate at 'correctness'

−2 / +127 lines of 1967 · 3 parallel region(s) added · 29 of 49 candidate(s) kept · RESTRUCTURING (94 code line(s) added, 2 removed) · restructuring level 0

BEFORE

```
1958      LagrangeNodal(domain);
1959
1960   #ifdef SEDOV_SYNC_POS_VEL_LATE
1961   #endif
1962
1963      LagrangeElements(domain, domain.numElem());
1964
1965      CalcTimeConstraintsForElems(domain);
```

ATTEMPT 4 OF 49, REJECTED BY THE GATE AT 'CORRECTNESS'

```
1958      LagrangeNodal(domain);
1959
1960   #ifdef SEDOV_SYNC_POS_VEL_LATE
1961   #endif
1962
1963      /* Inline LagrangeElements -> CalcLagrangeElements with parallelization */
1964      Index_t numElem = domain.numElem() ;
1965      if (numElem > 0) {
1966         const Real_t deltatime = domain.deltatime() ;
1967
1968         domain.AllocateStrains(numElem);
1969
1970         /* Inline CalcKinematicsForElems loop - parallelized for coarse granular
1971         #pragma omp parallel for schedule(static)
1972         for( Index_t k=0 ; k<numElem ; ++k )
1973         {
1974            Real_t B[3][8] ;
1975            Real_t D[6] ;
1976            Real_t x_local[8] ;
1977            Real_t y_local[8] ;
1978            Real_t z_local[8] ;
1979            Real_t xd_local[8] ;
1980            Real_t yd_local[8] ;
1981            Real_t zd_local[8] ;
1982            Real_t detJ = Real_t(0.0) ;
1983
1984            Real_t volume ;
1985            Real_t relativeVolume ;
1986            const Index_t* const elemToNode = domain.nodelist(k) ;
1987
1988            CollectDomainNodesToElemNodes(domain, elemToNode, x_local, y_local, z
1989
1990            volume = CalcElemVolume(x_local, y_local, z_local );
1991            relativeVolume = volume / domain.volo(k) ;
1992            domain.vnew(k) = relativeVolume ;
1993            domain.delv(k) = relativeVolume - domain.v(k) ;
1994
1995            domain.arealg(k) = CalcElemCharacteristicLength(x_local, y_local, z_l
1996                                                    volume);
1997
1998            for( Index_t lnode=0 ; lnode<8 ; ++lnode )
1999            {
2000               Index_t gnode = elemToNode[lnode];
2001               xd_local[lnode] = domain.xd(gnode);
2002               yd_local[lnode] = domain.yd(gnode);
2003               zd_local[lnode] = domain.zd(gnode);
2004            }
2005
2006            CalcElemShapeFunctionDerivatives( x_local, y_local, z_local,
2007                                              B, &detJ );
2008
2009            CalcElemVelocityGradient( xd_local, yd_local, zd_local,
2010                                      B, detJ, D );
2011
2012            domain.dxx(k) = D[0];
2013            domain.dyy(k) = D[1];
2014            domain.dzz(k) = D[2];
2015         }
2016
2017         /* Implicit barrier here, then vdov loop - still parallelized */
2018         bool volume_error = false;
2019         #pragma omp parallel for schedule(static) reduction(||:volume_error)
2020         for ( Index_t k=0 ; k<numElem ; ++k )
2021         {
2022            Real_t vdov = domain.dxx(k) + domain.dyy(k) + domain.dzz(k) ;
2023            Real_t vdovthird = vdov/Real_t(3.0) ;
2024
2025            domain.vdov(k) = vdov ;
2026            domain.dxx(k) -= vdovthird ;
2027            domain.dyy(k) -= vdovthird ;
2028            domain.dzz(k) -= vdovthird ;
2029
2030            if (domain.vnew(k) <= Real_t(0.0))
2031            {
2032               volume_error = true ;
2033            }
2034         }
2035         if (volume_error) {
2036            exit(VolumeError);
2037         }
2038
2039         domain.DeallocateStrains();
2040      }
2041
2042      /* Complete LagrangeElements with remaining sequential calls */
2043      CalcQForElems(domain) ;
2044      ApplyMaterialPropertiesForElems(domain) ;
2045      UpdateVolumesForElems(domain, domain.v_cut(), numElem) ;
2046
2047      /* CalcTimeConstraintsForElems inlined and parallelized */
2048      Real_t dtcourant = 1.0e+20;
2049      Real_t dthydro = 1.0e+20;
2050
2051      Real_t qqc2 = Real_t(64.0) * domain.qqc() * domain.qqc() ;
2052      Real_t dvovmax = domain.dvovmax() ;
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is